BITBOX-E2E

Every PR gets an automatic e2e run with device screenshots as evidence

One command builds the PR's firmware simulator, picks the scenarios that reach the changed code, shows the maintainer exactly which device screens changed, and flags anything that broke in pre-existing vectors and in the api-go / api-rs / api-ts suites. Native, no docker, about 1.5 minutes per PR.

89 s
full PR run, demo commit

28
units auto-selected from 1 changed function

10 / 10
identical graphical runs after flake fixes

$0
API cost; free runners on public repos

WHY

Today, checking a device change is manual

Hand testing on the simulator

Someone builds the simulator, clicks through the flow and eyeballs the screens. Nobody sees what the reviewer saw; there is nothing to attach to the PR.

Unit tests check strings, not flows

Firmware tests assert `Screen::` values against a mock UI. They don't render pixels, and they don't run the host API end to end.

Cross-repo drift goes unnoticed

Firmware changed the high-fee wording for 9.28.0 and updated its vectors. The copies in api-go, api-rs and api-ts were not updated. Their CI only tests released simulators, so it is green today and will break when 9.28.0 ships.

found by this harness

Prior art

Trezor (`--ui=test`, fixture hashes, HTML diff report) and Ledger (Speculos + Ragger golden PNGs) both do this. BitBox had the pieces (simulators, vectors) but not the pipeline.

HOW IT WORKS

One PR run, two lanes

PR
firmware, API or app change

→

Select
coverage map: changed functions → units that execute them

→

Lane A · evidence
run selected units on the head simulator, diff screens against the merge-base *baseline bundle* (text + LCD pixels)

Lane B · regression
check every unit with expectations; run api-go / api-rs / api-ts against the head simulator, with their own and the firmware's vectors

→

Report
report.html + PR comment: changed screens, failures, unreached code

Units

A unit is one scripted host session: a scenario in `scenarios/*.json` or a firmware BTC vector. Each runs headless (exact screen text) and, when its screens changed, on the graphical simulator (real LCD pixels, driven by touch gestures).

Baseline bundle

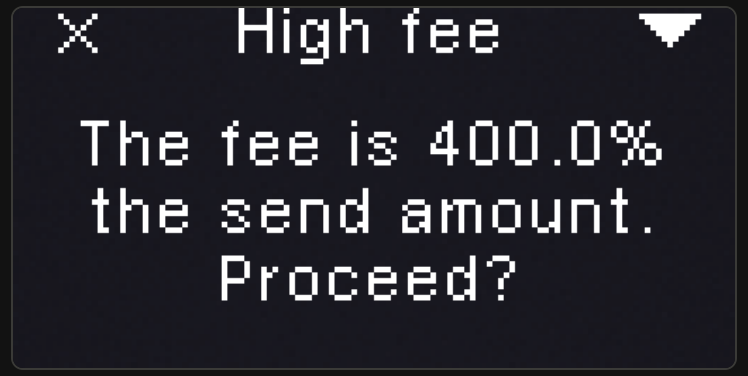
Built once per master commit: every unit's screens, frames, API results and the coverage map. PR runs reuse it, so only the head is built and run.

Expectations are optional

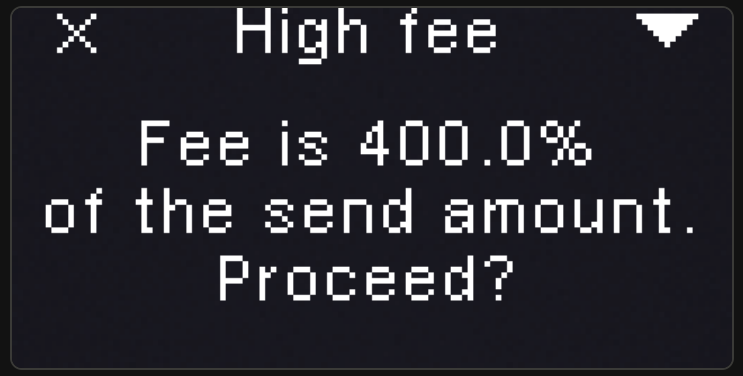
Without them, lane A still shows what changed. A maintainer can `promote` approved screens; from then on they are a regression check.

LANE A · REAL COMMIT D5FA70250 "FIX HIGH-FEE WARNING WORDING"

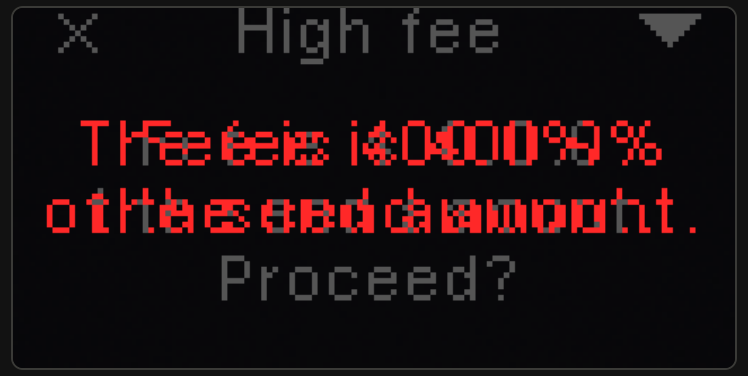
The maintainer sees the changed screen, not a log



base b59b48983



head d5fa70250



changed pixels in red

```
btc-vector/taproot-key-spend · step sign · CONFIRM "High fee"   (+27 more vectors, 6 distinct diffs)
- The fee is 400.0%\nthe send amount.\nProceed?
+ Fee is 400.0%\nof the send amount.\nProceed?
```

Selection picked the 28 BTC vectors that reach `warning_message`; nothing changed was left unreached. Graphical runs only for one unit per distinct diff.

LANE A · NEW SCREENS

New flows show up as added frames



added: "Token contract" confirmation for an ERC-20 transfer to an unknown contract



unchanged: next screen, also present on base

```
scenario/eth-erc20-unknown · step erc20-transfer
  CONFIRM  Sign transaction on Ethereum
+ CONFIRM  Token contract
+          0x 9c23 D67a eA7b 95D8 0942 e383 6BCD f7E7 08a7 47C1
  CONFIRM TRANSACTION ADDRESS  Unknown token
           0x E6CE 0a09 2A99 700C ...
```

The text transcript is authoritative (full addresses, no scrolling); the PNGs are what the user actually sees. Device setup (pairing code, seeding, clock) is kept out of the diff.

LANE B · REGRESSION

The same run flagged real drift in the API repos

SUITE	HEAD, REPO'S OWN VECTORS	HEAD, FIRMWARE'S VECTORS	BASE (BOTH)
api-go	**50 pass / 31 fail**	81 pass	81 pass
api-rs	**52 pass / 1 fail**	53 pass	53 pass
api-ts	**25 pass / 1 fail**	26 pass	26 pass

Reading it

The API repos carry copies of `btc-transaction-test-vectors.json` with the old wording. Against a 9.28.0 simulator their own suites fail; with the firmware's current vectors they pass. Fix: refresh the copies before 9.28.0 is added to their CI.

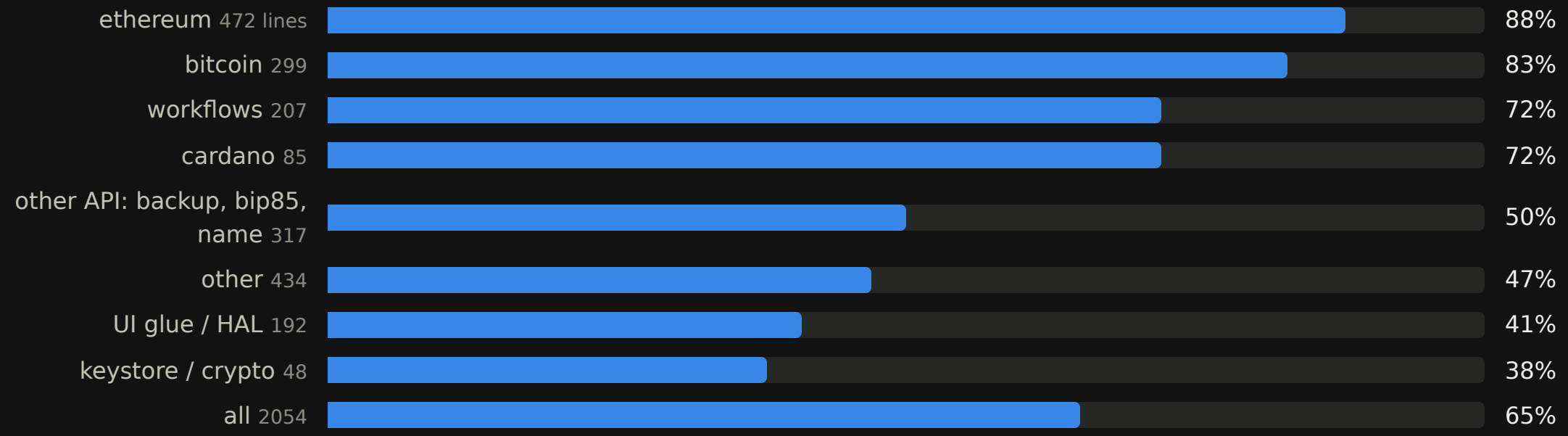
Expectations

All 28 selected vectors pass the 9.28.0 expectations on head. On the base commit the same 28 fail, which is exactly the change under review.

COVERAGE

How much new firmware code do existing suites run?

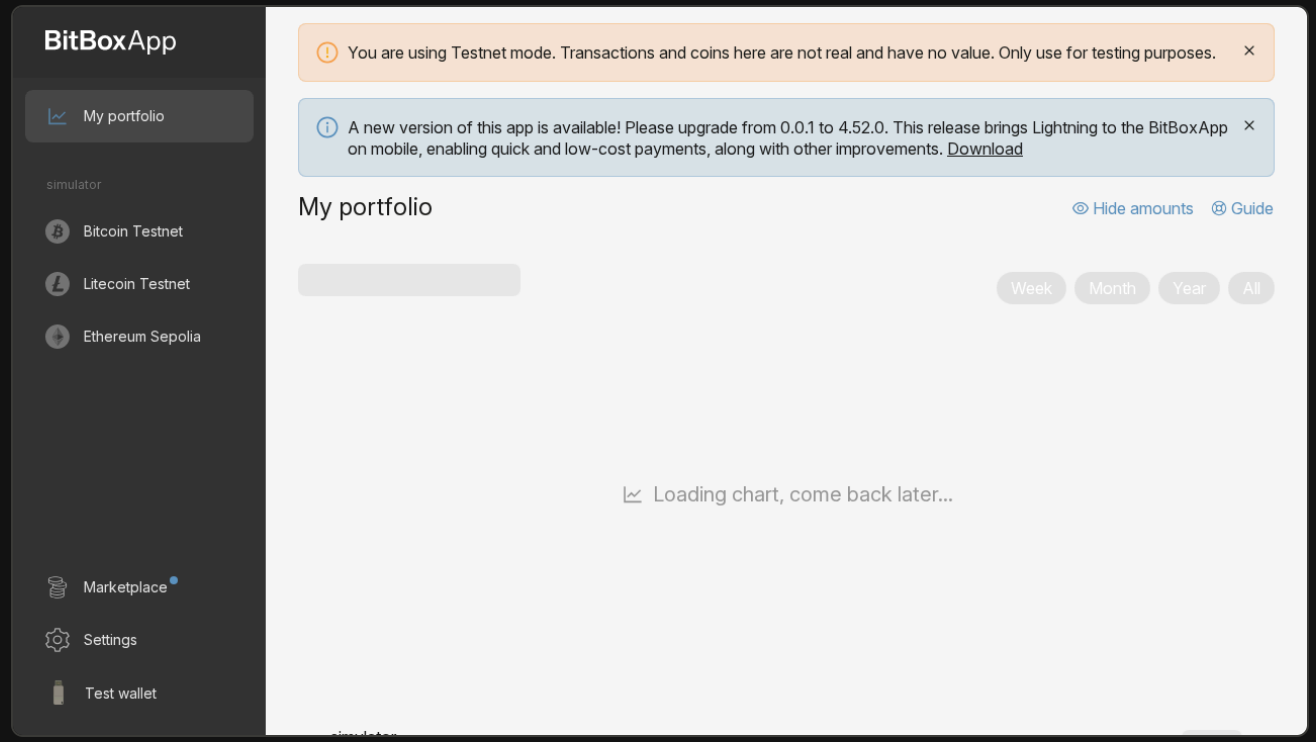
Executable Rust lines added on master in the last 6 months that are linked into the simulator, and the share run by any existing suite (api-go, api-rs, api-ts, e2e scenarios). Measured with an instrumented simulator.



Each PR report lists the changed functions no unit reaches, so the gaps are visible per change. Biggest today: unlock, device language / i18n, bip85, backup. C UI code is not instrumented.

BITBOXAPP

App changes: Playwright against the same simulator



watch-only.test.ts, head, Chromium, simulator device connected

`e2e.py wallet-compare` runs the app's own Playwright tests from clean clones of base and head, with a natively built simulator. The report lists per-test status, errors, device screen diffs and final screenshots that differ.

Next: masked screenshots at key checkpoints as expectations (Trezor/Ledger style), and a coverage map from V8 + Go coverage so app PRs get selection too.

CONTRIBUTOR WORKFLOW

No expectations to write up front

1 · Open the PR

CI runs `e2e.py pr` against the merge-base bundle and posts the report.

2 · Unreached code?

The report lists changed functions no unit reaches. The `bitbox-e2e` Claude skill drafts a scenario (or a driver op) for them, runs it locally and attaches the evidence.

3 · Maintainer reviews screens

Approve what the device shows, or ask for changes, from the report.

4 · Promote

`e2e.py promote <out> scenario/x` writes the approved screens as a version-ranged expectation.

```
<!-- e2e   optional PR-body block (planned)
units: btc-vector/*fee*, scenario/eth-erc20-*
graphical: btc-vector/*fee*
api-rs: my-branch   # unreleased API call
-->
```

Adds to the automatic selection, never replaces it. Useful for showcase screenshots of a new flow, or when the driver needs an unreleased bitbox-api-rs branch.

COST

Minutes per PR, no paid APIs

PHASE (DEMO PR, 8 CORES)	TIME
baseline bundle (reused)	0 s
head simulator build (cached by sha)	0 s · ~4 min cold
28 units headless, 4 parallel	12 s
6 units graphical	40 s
lane B: 3 API suites × 2 vector sets	37 s
total	**89 s**

CI

- Standard GitHub runners, free on public repos; expect ~2× on 4 vCPU
- No GPU, no hardware: Xvfb + software GL
- Baseline + coverage map per master merge: ~10–15 min (two simulator builds dominate)
- Storage: a few MB per bundle; app videos only on failure

Not covered

- Hardware-only code (~15% of firmware commits): bootloader, BLE, secure chip
- BitBox03 UI until its graphical simulator is wired in

USE IT

Commands

```
# once per machine (no root): LLVM, protoc, Rust pin, node, driver
e2e/setup.sh

# a PR, against its merge-base; WORKTREE = uncommitted firmware checkout
e2e/e2e.py pr --base bitbox/master --head my-branch --out /tmp/e2e/my-pr

# baseline bundle + coverage map for a master commit (CI does this per merge)
e2e/e2e.py baseline bitbox/master
e2e/coverage.py map bitbox/master

# what a diff reaches, and what it doesn't
e2e/coverage.py select --base bitbox/master --head my-branch --summary

# accept reviewed screens as expectations
e2e/e2e.py promote /tmp/e2e/my-pr scenario/eth-erc20-unknown

# app change
e2e/e2e.py wallet-compare --base bitbox/master --head my-branch --firmware bitbox/master \
  --out /tmp/e2e/app tests/watch-only.test.ts
```

STATUS

Prototype works end to end; next is CI

Done

- Native simulator builds, cached by sha
- Units: scenarios + 37 firmware BTC vectors, optional expectations, promote
- Baseline bundles, `pr` with lanes A and B, HTML report
- Coverage map, selection, unreached-function list
- Parallel isolated runs; graphical runs deterministic (10/10)
- App compare via Playwright
- Claude skill for running it and drafting scenarios

Next

- Reusable GitHub workflow + PR comment; bundles as release assets
- ETH vector family in firmware, next to the BTC vectors
- Scenarios for the biggest gaps: unlock, language, bip85, backup
- Parse the PR-body `e2e` block; api-rs branch override
- App: masked checkpoint screenshots, coverage-based selection
- Refresh the stale vector copies in api-go / api-rs / api-ts

Repo: `github.com/Tomasvrba/bitbox-e2e` (private for now, move to the org later)